

# Lecture 8: Finish additive models, classification

Max G'Sell  
Machine Learning II: 46-927

November 23, 2020

# Announcements

- ▶ No homework due this week, no quiz this week.  
There is a two-week homework due next week. It's out now, but I reserve the right to modify the end (problem 4, unlikely problem 5) through tomorrow depending on how today goes.

At the very least, make sure that you can get pygam working early, just in case there are software difficulties.

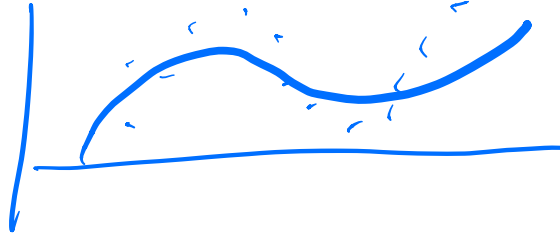
- ▶ (I haven't figured out how to fix the computed grades for that question from quiz 2 yet... but we will.)

# Today

Today we will mostly be changing gears to classification and discussing basic theory along with discriminative models like logistic regression.

- ▶ Brief wrap-up of additive models
- ▶ Classification
- ▶ Logistic regression

# Smoothing splines



We saw that, as an alternative to kernel regression, we can fit smooth, non-linear functions using smoothing splines:

$$\hat{f} = \operatorname{argmin}_f \sum_{i=1}^n (y_i - \overbrace{f(x_i)}^{x^T \beta})^2 + \lambda \underbrace{\int f''(x)^2 dx}$$

This gives a computationally efficient way to swap smooth, flexible functions in for linear functions throughout our methods.

# Now what?

So far, we've seen that:

- ▶ We can build nonlinear function estimates that converge to the true regression function with enough data (and smoothness).
- ▶ We can make them efficient in one dimension, using splines.
- ▶ Our kernel methods break down in higher dimensions.

We don't usually just care about one-dimensional regression! What can we do?

~

# More than one dimension

Suppose that we want to estimate  $Y$  from  $X \in \mathbb{R}^p$ , with  $p > 1$ . 

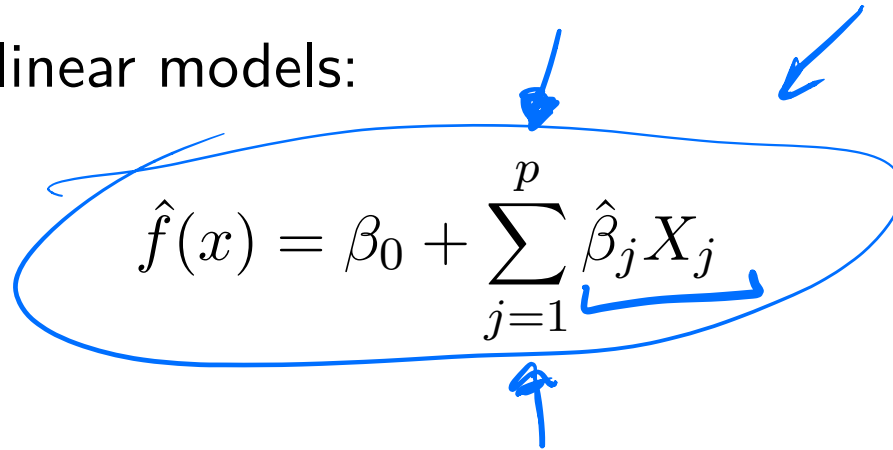
For quite small  $p$ , we could still use kernel regression. There are also two generalizations of splines to low dimensions, but they suffer from the same curse.

We look for a compromise: more flexibility than linear regression, but less than general  $d$ -dimensional function estimation.


# Additive models

Think about our linear models:



The equation  $\hat{f}(x) = \beta_0 + \sum_{j=1}^p \hat{\beta}_j X_j$  is enclosed in a blue oval. A blue arrow points from the top right towards the equation. Another blue arrow points from the top left towards the equation. A third blue arrow points from the bottom towards the equation. A fourth blue arrow points from the right towards the term  $\hat{\beta}_j X_j$  in the summation.

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^p \hat{\beta}_j X_j$$

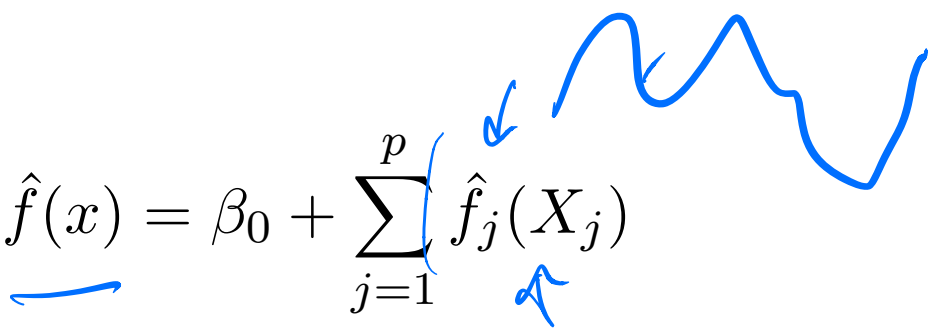
There are really two modeling choices going into this:

- ▶ Additivity: Each feature contributes additively to the prediction.
- ▶ Linearity: The values of each feature enter linearly.

It turns out that linearity can be relaxed without sacrificing much efficiency!

# Additive models

Additive models:

$$\hat{f}(x) = \beta_0 + \sum_{j=1}^p \hat{f}_j(X_j)$$
Hand-drawn blue arrows pointing to the components of the equation: one arrow points to the entire equation, another points to the summation symbol, a third points to the index  $j$ , a fourth points to the function  $\hat{f}_j$ , and a fifth points to the feature  $X_j$ . Additionally, a blue wavy line is drawn above the summation.

We can choose the  $\hat{f}_j$  in many different ways. We can even vary the choice for each feature!

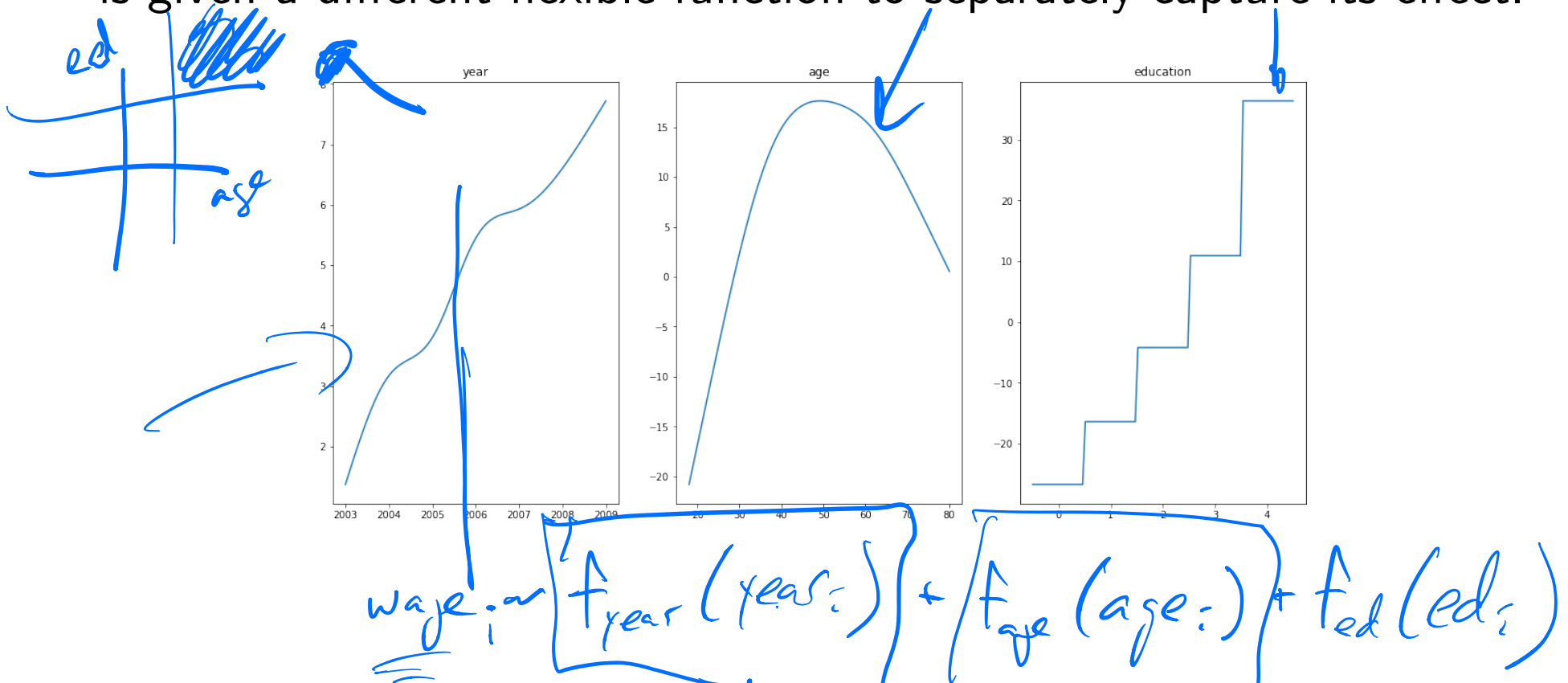
- ▶ Linear function
- ▶ Polynomial function
- ▶ Smoothing spline 
- ▶ Kernel regression
- ▶ Local linear fit

In practice, fitting smoothing splines for each  $\hat{f}_j$  is most common. This is what the `LinearGAM` function in python does (package `pygam`, and what `gam`, `mgcv` packages do in R. (*Note: mgcv is the easiest.*))

# Additive models

$$\min_{f_1, \dots, f_p} \sum_{i=1}^n \left( y_i - \beta_0 - \sum_{j=1}^p f_j(x_{ij}) \right)^2 + \sum_{j=1}^p \lambda_j \int \left( f_j''(x) \right)^2 dx$$

We're actually minimizing a problem like this, where each variable is given a different flexible function to separately capture its effect.



## Additive models

To estimate a function linearly, we expect

$$MSE = O\left(\frac{1}{n}\right)$$

To estimate a 1-dimensional function nonparametrically, we expect

$$MSE = O\left(\frac{1}{n^{4/5}}\right)$$

To estimate a  $d$ -dimensional function nonparametrically, we expect

$$MSE = O\left(\frac{1}{n^{4/(\underline{d+4})}}\right)$$

## Additive models

To estimate a  $d$ -dimensional function nonparametrically, we expect

$$MSE = O\left(\frac{1}{n^{4/(d+4)}}\right)$$

To estimate a  $d$ -dimensional *additive* model, we essentially just estimate each piece separately, giving

$$MSE = O\left(\frac{1}{n^{4/5}}\right)$$

This means that, if we don't care about this minor drop in convergence rate, we can replace linear regression with non-linear functions of each variable!

No more guessing at variable transformations by hand!

## What's next

We've talked about several topics for prediction, starting with linear regression and moving in both lower variance and lower bias directions.

We're going to change gears and talk about classification, the other major chunk of supervised learning.

After we're comfortable with both of these areas, we're going to talk about more advanced modeling techniques that are useful for both prediction and classification.

# How about classification?

Recall that classification refers to problems where you see  $X \in \mathbb{R}^p$ , and try to guess  $y \in \{1, \dots, K\}$  using a classification function  $f(X)$ . This function is usually estimated from training data.

You may have seen several methods: Logistic regression, Generalized additive logistic models, Support Vector Machines (SVM), Nearest Neighbors, Naive Bayes, Decision Trees, Random Forests

We will mostly focus on two-class problems ( $K = 2$ ), and we will often denote the classes as  $\{0, 1\}$  or  $\{-1, 1\}$ .

# Confusion matrix

- ▶  $Y = 1$  if an event of interest happened
- ▶  $Y = 0$  if it did not happen

In a classification problem, we can make  $K(K - 1)$  different kinds of mistakes!

This leads to the *confusion matrix*, which tabulates guessed categories against true categories.

In our  $2 \times 2$  setting:

|                                 | True <u><math>Y = 1</math></u> | True <u><math>Y = 0</math></u> |
|---------------------------------|--------------------------------|--------------------------------|
| Guess <u><math>Y = 1</math></u> | True Positives                 | False Positive                 |
| Guess $Y = 0$                   | False Negative                 | True Negative.                 |

In general, there is one entry for each class in the rows and columns.

## Confusion matrix: error rates

|               | True $Y = 1$   | True $Y = 0$   |
|---------------|----------------|----------------|
| Guess $Y = 1$ | True Positive  | False Positive |
| Guess $Y = 0$ | False Negative | True Negative  |

We are interested in many different quantities in the confusion matrix. Probably the most famous:

► Misclassification rate:

$$\frac{FP + FN}{n}$$


► Accuracy:

$$1 - \frac{FP + FN}{n}$$


If False positives and False negatives are equally bad, then minimizing the misclassification rate is a good goal.

*This is not always a good idea! We will come back to this later.*

## Cost/Loss for classification

When building classifiers, it is useful to think of the confusion matrix in terms of *losses*. This is analogous to loss functions in regression.

Because our guesses and outcomes are discrete, the loss function can be summarized by a matrix:

|               | True $Y = 1$ | True $Y = 0$ |
|---------------|--------------|--------------|
| Guess $Y = 1$ | $L_{11}$     | $L_{01}$     |
| Guess $Y = 0$ | $L_{10}$     | $L_{00}$     |

Usually we think of  $L_{11} = L_{00} = 0$ .

In most cases,  $L_{01}$  (cost of guessing  $Y = 1$  when it is actually  $Y = 0$ ) is quite different from  $L_{10}$ .

However, the simplest loss, 0-1 loss, sets  $L_{ij} = 1$  for  $i \neq j$ .

# 0-1 Loss

The 0-1 loss is unrealistic for many scenarios, since the cost for errors is usually asymmetric.

However, it gives us a good starting point for developing methods.

The methods that result from 0-1 loss can often be tweaked to give good results for other losses.

Suppose that we decide to minimize 0-1 loss. How should we carry it out?

# Statistical decision theory

$\mathcal{P}_{\text{known}}$

Ideally, we would choose our prediction  $\hat{f}(X)$  to minimize the expected loss (also called **expected prediction error** or **expected misclassification error**).

$$\begin{aligned} \text{EPE} &= \mathbb{E}(\mathcal{L}(Y, \hat{f}(X))) = E\left[E(\mathcal{L}(Y, \hat{f}(x)) | X)\right] \\ &= E\left[\sum_{k=1}^K \underbrace{P(Y=k|X)}_{\text{conditioned on } X} \underbrace{\mathcal{L}(k, \hat{f}(x))}_{\text{loss}}\right] \end{aligned}$$

We conditioned on  $X$ , so that we can first take the expectation over  $Y$  with  $X$  held fixed, and then take the expectation over  $X$ .

# Statistical decision theory

$\downarrow \mathcal{L}^{\mathbb{R}^P} \rightarrow \{1, \dots, K\}$

Now we minimize the inside of the expectation over  $\hat{f}(X)$  for each possible value of  $X$ , pointwise: 0-1 Loss

$$\begin{aligned}
 \hat{f}(x) &= \operatorname{argmin}_{g \in \{1, \dots, K\}} \sum_{k=1}^K \mathcal{L}(k, g) \mathbb{P}(Y = k | X = x) \\
 &= \operatorname{argmin}_g \sum_{k=1}^K \mathbb{1}_{g \neq k} P(Y = k | X = x) \\
 &= \operatorname{argmin}_g \sum_k (1 - \mathbb{1}_{g=k}) P(Y = k | X = x) \\
 &= \operatorname{argmin}_g \left[ 1 - \sum_k (\mathbb{1}_{g=k} \cdot P(Y = k | X = x)) \right] \\
 &= \operatorname{argmin}_g 1 - P(Y = g | X = x) \\
 &= \operatorname{argmax}_g P(Y = g | X = x)
 \end{aligned}$$

# Statistical decision theory

The decision theory framework leads us to a natural rule for classification:

$$\hat{f}(x) = \operatorname{argmax}_{j=1,\dots,K} \underbrace{P(Y = j | \underline{X = x})}$$

This predicts the most likely class, given the feature measurements  $X = x \in \mathbb{R}^p$ .

This is called the Bayes classifier, and it achieves the best possible expected misclassification error (if we could do it)

↑ Bayes Rate

# Statistical decision theory

Remember Bayes theorem, which says:

$$P(Y = j | X = x) = \frac{P(X = x | Y = j) P(Y = j)}{P(X = x)}$$

Handwritten annotations: A red arrow points from  $\pi_j$  to  $P(Y = j)$ . A red bracket underlines  $P(Y = j | X = x)$ . A red arrow points from the top of the fraction bar to the denominator  $P(X = x)$ .

[Note:  $P(X = x | Y = j)$  and  $P(X = x)$  could also be densities.]

Let  $\pi_j = P(Y = j)$  be the **prior probability** of class  $j$ .

Since the Bayes classifier compares the above quantity across  $j = 1, \dots, K$  for  $X = x$ , the denominator is always the same, hence

$$\hat{f}(x) = \operatorname{argmax}_{j=1, \dots, K} P(X = x | Y = j) \cdot \underline{\underline{\pi_j}}$$

# Building a classifier

$$\begin{aligned}\hat{f}(x) &= \operatorname{argmax}_{j=1,\dots,K} P(Y = j|X = x) \\ &= \operatorname{argmax}_{j=1,\dots,K} P(X = x|Y = j) \cdot \pi_j\end{aligned}$$

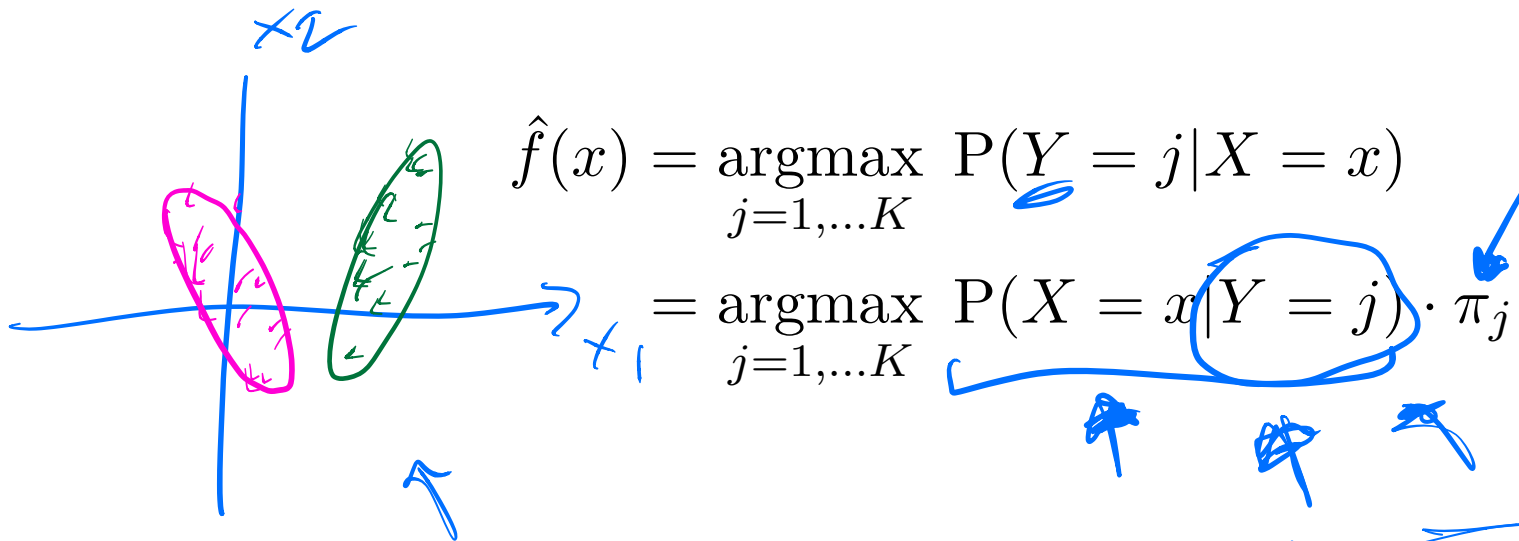
How do we actually use this to build a classifier?

1. We can try to estimate  $\mathbb{P}(Y = j|X = x)$  directly.

Logistic regression is a parametric version of this, while k-Nearest neighbors is a non-parametric version of this.

# Building a classifier

$x \in \mathbb{R}^p$



How do we actually use this to build a classifier?

*Generative Models.*

2. We could try to estimate the distributions  $\mathbb{P}(X = x | Y = j)$  for each class. That is, given that an observation comes from class  $j$ , how are its covariates distributed?

Linear discriminant analysis (LDA) and naive Bayes are both classic examples of this. We will come back to these later.